Day 03 - Docker Volumes & Bind Mounts

Objectives

Understand persistent storage, Docker Volumes, Bind Mounts, and data persistence.

Topics Covered

- Writable Layer vs Volume
- Docker Volume lifecycle
- Volume commands
- Mounting volumes
- Sharing volumes
- Docker Volume vs Bind Mount
- Production vs Development use cases

Important Commands

```
docker volume create my-volume
docker volume ls
docker volume inspect my-volume
docker volume rm my-volume

docker run -d --name nginx-volume -p 8080:80 -v my-volume:/usr/share/nginx/html nginx
docker exec -it nginx-volume sh
docker rm -f nginx-volume
```

Workflow

Create Volume → Run Container → Mount Volume → Modify Data → Delete Container → Volume Persists → Create New Container → Data Restored

Docker Volume vs Bind Mount

Docker Volume: Docker-managed storage, ideal for databases and production.
Bind Mount: Maps a host directory into a container, ideal for live development.

Key Interview Points

• Volumes survive container deletion.
• Multiple containers can share a volume.
• Use Volumes for MySQL/PostgreSQL/MongoDB.
• Use Bind Mounts for Node.js/React development.

Key Takeaways

Containers are ephemeral. Writable layers are temporary. Docker Volumes provide persistent storage independent of containers.